

---

## Contents

|                                        |          |
|----------------------------------------|----------|
| <b>RISC-V Instruction Set</b>          | <b>1</b> |
| Storia RISC-V . . . . .                | 1        |
| Caratteristiche RISC-V ISA . . . . .   | 2        |
| Registri in RISC-V . . . . .           | 2        |
| Estensioni RISC-V ISA . . . . .        | 3        |
| Struttura comando RISC-V ISA . . . . . | 4        |
| Formato tipo R . . . . .               | 4        |
| Formato tipo I . . . . .               | 4        |
| Formato di tipo S . . . . .            | 5        |
| Formato di tipo SB . . . . .           | 6        |
| Formato di tipo J . . . . .            | 6        |

## RISC-V Instruction Set

### Storia RISC-V

L'ISA di RISC-V nasce all'università di Berkeley nel 2010 da Andrew Waterman, Yunsup Lee, Krste Asanovic. L'obiettivo era costruire attraverso uno standard aperto, hardware semplice in grado di:

- Massimizzare le prestazioni
- Minimizzare i costi
- Minimizzare i consumi

L'ISA di RISC-V è simile a MIPS e a Intel x86 questo perchè sono costruiti tutti sugli stessi principi fondamentali cioè le operazioni di base di un ISA. Inoltre l'ISA deve svolgere istruzioni semplici e di base.

**RISC-V è uno standard** mantenuto dalla RISC-V foundation ed è adatto ad una vasta gamma di dispositivi (da microcontrollori ai supercomputer).

RISC significa Reduced Instruction Set Computer e porta i seguenti benefici:

- La semplicità favorisce la regolarità
- Minori sono le dimensioni, maggiore è la velocità
- Un buon progetto richiede buoni compromessi

---

## Caratteristiche RISC-V ISA

Il RISC-V ISA ha le seguenti caratteristiche:

- 32 registri(x0-x31). Contengono gli operandi per poter eseguire operazioni. x0 contiene sempre il valore 0
- $2^{61}$  parole di memoria. RISC-V usa l'indirizzamento al byte. Questa memoria serve per memorizzare strutture dati, vettori o il contenuto dei registri. **Non può essere usata dagli operandi**
- Gli interi sono memorizzati in complemento a due ### Memoria e Registri Un parola è detta word ed è composta da 32 bit(una sequenza di 4 bytes). Una doubleword è lunga 64 bit(una sequenza di 8 bytes). Esiste anche halfword cioè 16 bit(2 bytes)

Inoltre i registri hanno la seguente composizione:

- 32 registri sono per gli interi(a 32 o 64bit in base all'estensione)
- 32 registri sono per i numeri in virgola mobile(a 32 o 64bit in base all'estensione)
- I registri sono grandi 32 o 64 bit in base all'estensione
- Abbiamo il registro PC program counter
- Ci sono 4096 control status registers
- Possiamo indirizzare la RAM per  $2^n-1$  dove n è l'estensione in bit che stiamo usando(32 o 64). Ogni cella di memoria è grande 8 bit. Gli indirizzi si misurano con i byte

## Registri in RISC-V

| Registro | RARS | Uso                                                                                                                                                     |
|----------|------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| x0       | zero | Costante 0. Read-Only. Dato che viene usata spesso è disponibile hardwired come registro                                                                |
| x1       | ra   | return address(ra). Cioè l'indirizzo dove viene chiamata una funzione, viene usato per tenere traccia del punto dove ritornare alla fine della funzione |
| x2       | sp   | Stack pointer(sp). L'indirizzo alla fine dello stack per la funzione corrente                                                                           |
| x3       | gp   | Global pointer(gp)                                                                                                                                      |
| x4       | tp   | Thread pointer(tp)                                                                                                                                      |

---

---

| Registro       | RARS             | Uso                                                                                                                                                                     |
|----------------|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| x5-x7, x28-x31 | t0-t2,<br>t3-t6  | Registri temporanei. Non salvano i valori in caso di chiamata di una procedura. Sono ad utilizzo generico                                                               |
| x8             | fp               | Frame pointer(fp). L'indirizzo all'inizio del frame della funzione corrente oppure registro da salvare                                                                  |
| x8-x9, x18-x27 | s0-s1,<br>s2-s11 | Registri da salvare: Il contenuto va preservato se utilizzati dalla procedura chiamata                                                                                  |
| x10-x17        | a0-a7            | Registri usati per il passaggio di parametri nelle procedure e per i valori di ritorno. Se necessario(e non sono utilizzati) possono anche essere usati ad uso generico |

---

Sono tutti uguali a livello fisico, tuttavia non possono essere usati tutti allo stesso modo(ad esempio non possiamo usare liberamente lo stack pointer o il frame pointer oppure usare x0, ecc...)

Esistono anche degli alias in RARS per facilitarci(seconda colonna).

## Estensioni RISC-V ISA

Il RISC-V ISA può anche essere esteso. Le seguenti estensioni standard sono state portate dal 2010 ad oggi:

---

| Extension | Description                            |
|-----------|----------------------------------------|
| I         | Integer                                |
| M         | Moltiplicazioni tra interi e divisioni |
| F         | Single precision floating point        |
| D         | Double precision floating point        |
| C         | 16-bit compressed instructions         |

---

Esistono anche estensioni non standard chiamate *Extensions*

---

Inoltre esistono anche delle varianti dell'ISA:

- RV32I: Instruction set a 32 bit(base integer 32 bit instruction set)
- RV64I: Instruction set a 64 bit(base integer 64 bit instruction set)
- RV64M: Instruction set con istruzioni per moltiplicazioni e divisioni intere
- RV64F: Instruction set con istruzioni per la gestione di numeri a virgola mobile
- RV64E/RV32E: Instruction set con 16 registri

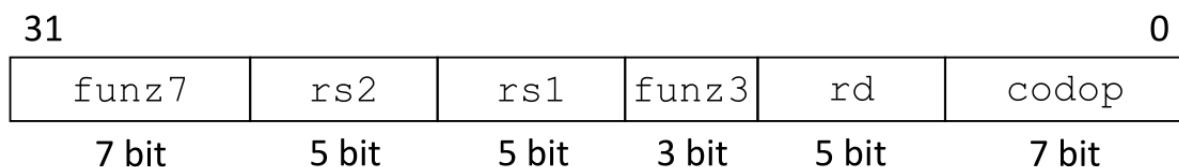
## Struttura comando RISC-V ISA

Le istruzioni assembly vanno convertite in linguaggio macchina, le CPU RISC-V devono decodificare(fetch-decode) le istruzioni che vengono passate come sequenze di 32 bit.

### Formato tipo R

Questo formato viene usato per le istruzioni che operano direttamente su 3 registri come add, sub, or, ...

Le struttura di un'istruzione RISC-V di tipo R(registro) è



**Figure 1:** alt text

I campi dell'istruzione sono: - 7 bit codop: codice operativo dell'istruzione - 5 bit rd: registro di destinazione - 5 bit rs1: registro che contiene il primo operando sorgente - 5 bit rs2: registro che contiene il secondo operando sorgente - 3 bit funz3, 7 bit funz7: codici operativi aggiuntivi

**I registri sono distinti su 32 bit( $2^5 = 32$  registri)**

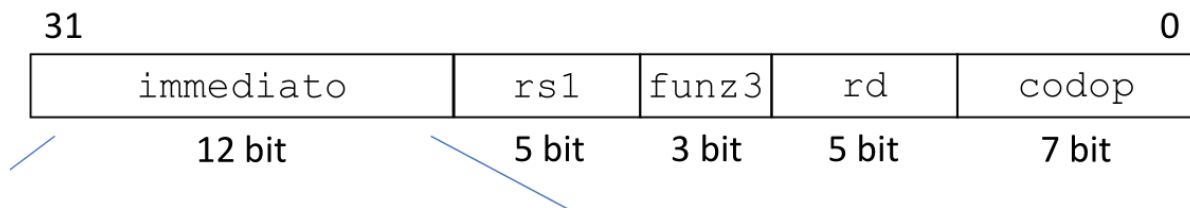
**L'istruzione viene identificata da codop+funz3+funz7.** Ad esempio add e sub differiscono di un bit in funz7.

### Formato tipo I

Questo formato viene usato per le istruzioni di tipo immediato come addi, subi, ori, ...

---

Le struttura di un'istruzione RISC-V di tipo I(immediato) è



**Figure 2:** alt text

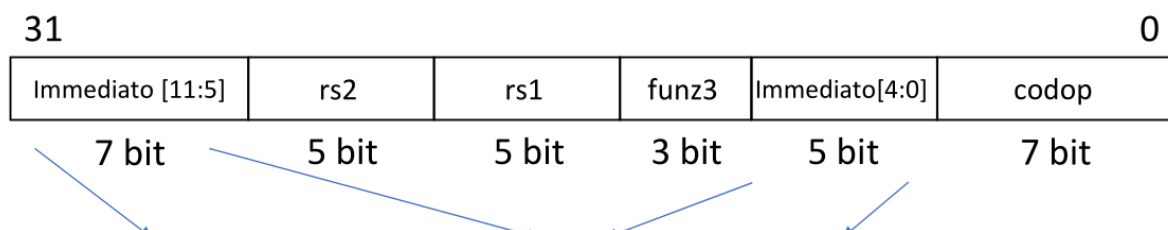
I campi dell'istruzione sono: - 7 bit codop: codice operativo dell'istruzione - 5 bit rd: registro di destinazione - 3 bit funz3: codici operativi aggiuntivi - 5 bit rs1: Registro che contiene il primo operando sorgente - 12 bit immediato: Numero su 12 bit in complemento a due(-2048 a +2047)

### Formato di tipo S

Questo formato viene usato per le istruzioni di tipo store come lw, lbu, ...

è differente dalle altre perchè richiede due registri e un offset.

Le struttura di un'istruzione RISC-V di tipo S(store) è



**Figure 3:** alt text

I campi dell'istruzione sono: - 7 bit codop: codice operativo dell'istruzione - 5 bit di immediato. I primi 5 bit meno significativi dell'offset - 3 bit funz3: codici operativi aggiuntivi - 5 bit rs1: Registro che contiene il primo operando sorgente - 5 bit rs2: Registro che contiene il primo operando sorgente - 7 bit immediato: I 7 bit più significativi dell'offset

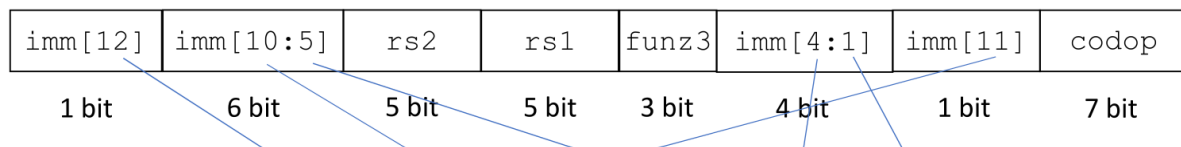
L'immediato è in complemento a due su 12 bit.

L'immediato viene diviso in due parti per far rimanere rs1 e rs2 nelle stesse posizioni degli altri formati così da rendere più efficiente la decodifica delle istruzioni(infatti se sono nelle stesse posizioni, sarà più veloce a decodificare)

---

## Formato di tipo SB

Questo formato viene usato per le istruzioni di tipo salto condizionato. è completamente differente dagli altri.



**Figure 4:** alt text

I campi dell'istruzione sono: - 7 bit codop: codice operativo dell'istruzione - 1 bit di immediato. b11 - 4 bit di immediato. b1 a 4 - 3 bit func3: codici operativi aggiuntivi - 5 bit rs1: Registro che contiene il primo registro sorgente - 5 bit rs2: Registro che contiene il secondo registro sorgente - 6 bit immediato da b5 a b10 - 1 bit immediato b12

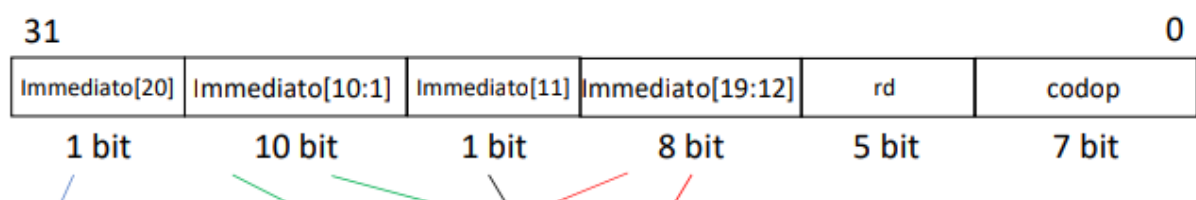
rs1 e rs2 sono nella stessa posizione degli altri formati.

I campi immediato sono formati da 13 bit(b1-b12 + b0) e rappresentano l'etichetta. Il bit 0 è sempre settato a 0. I valori possibili per questo immediato sono solo numeri pari(-4096 a +4094). Questo avviene perchè ogni istruzione è lunga 32 bit ma i progettisti avevano previsto anche l'estensione a 16 bit(quindi si può saltare solo a multipli di 2). **In 32 bit b1 è sempre settato a 0(insieme a b0)** così in 32 bit ci saranno solo indirizzi multipli di 4 mentre in 16 bit avremo anche multipli di 2 perchè si usa b1. I valori sono in complemento a due.

Questo immediato rappresenta l'indirizzamento relativo al program counter(PC-relative addressing). Cioè quando viene eseguito un salto viene eseguito  $PC = PC + \text{immediato}$  che porta il processore ad eseguire l'istruzione all'etichetta che abbiamo scelto.

## Formato di tipo J

Viene usato per le istruzioni jump(esempio **jal**).



**Figure 5:** alt text

---

I campi dell'istruzione sono: - 7 bit codop: codice operativo dell'istruzione - 5 bit che conterranno il registro destinazione dove verrà memorizzato il return address - 8 bit di immediato da b12 a b19 - 1 bit di immediato che rappresenta b11 - 10 bit di immediato da b1 a b10 - 1 bit di immediato che rappresenta b20

**I vari formati vengono riconosciuti dalla CPU attraverso il codop**